

DESAFIO DE AUTOMAÇÃO DIGITAL

Relatório Teórico: Controle de Qualidade e Triagem de Peças na Linha de Produção

1. Contextualização: Por que automatizar esse processo?

Quando comecei a analisar o problema proposto neste desafio, ficou muito claro o motivo de a indústria investir tanto em automação e abandonar as conferências manuais. No início dos estudos sobre manufatura e automação, a gente aprende que depender puramente da atenção de pessoas no chão de fábrica traz três problemas sérios:

- **Cansaço e erro humano:** Ficar medindo peças, conferindo peso e olhando cores repetidas vezes ao longo de um turno inteiro é exaustivo. Uma pessoa naturalmente perde a concentração depois de várias horas, o que aumenta muito a chance de uma peça com defeito passar batida e ir parar no cliente.
- **A esteira fica lenta (gargalo):** A velocidade de produção da fábrica acaba limitada pela rapidez com que o operador consegue inspecionar cada item na mão. Isso trava o fluxo da linha e gera filas de material parado.
- **Desperdício de material e retrabalho:** Se a inspeção demorar ou não notar um desvio logo no começo, centenas de peças podem ser fabricadas fora do padrão antes de alguém perceber. Isso vira sucata ou exige retrabalho, gerando um custo alto para a empresa.

Automatizar essa triagem resolve essas questões de vez. O programa aplica exatamente os mesmos critérios para todas as unidades, decide em frações de segundo e ainda registra o que aconteceu em cada peça, criando um histórico confiável de tudo o que foi aprovado ou reprovado.

2. Como pensei a lógica do algoritmo em Python

Para desenvolver o código, a ideia principal foi aplicar as boas práticas que estamos aprendendo nas primeiras aulas de programação, organizando tudo de maneira simples de ler e dar manutenção:

- **Separação por funções:** Em vez de colocar todo o código em um bloco único e gigante, dividi o programa em partes bem definidas. Uma função faz o teste de qualidade dos atributos da peça, outra cuida da contagem e do empacotamento nas caixas, e outra gera o resumo final com os números da produção.
- **Checagem completa com if/else:** Um detalhe importante na construção da lógica foi não interromper a conferência no primeiro defeito que o programa encontra. O algoritmo testa de forma independente o peso (95g a 105g), o comprimento (10cm a 20cm) e a cor (somente verde ou azul). Se a peça tiver dois problemas ao mesmo tempo — por exemplo, muito pesada e com a cor errada —, o sistema guarda os dois motivos numa lista. Dessa forma, quem olha o relatório sabe exatamente o que aconteceu de errado.
- **Controle das caixas com laço de repetição:** Utilizei uma estrutura for para percorrer a lista de peças. A lógica de armazenamento vai contando as unidades aprovadas e, quando atinge 10 itens na caixa atual, 'fecha' essa caixa e abre uma próxima para continuar guardando o restante.

- **Tratamento do texto digitado:** Para prevenir erros simples de digitação — como alguém digitar 'Azul' com maiúscula ou colocar um espaço sem querer no final —, usei métodos básicos do Python como `.strip()` e `.lower()`. Isso padroniza as palavras e evita rejeitar uma peça válida por bobeira de digitação.

3. O que deu certo e quais foram as maiores dificuldades

3.1. O que funcionou bem

- **Diagnóstico detalhado das falhas:** Como o código aponta exatamente qual regra foi violada, fica fácil entender em qual etapa do maquinário pode estar ocorrendo descalibração.
- **Separação correta das peças:** Apenas os itens 100% aprovados entram na contagem das caixas. Qualquer item que apresente pelo menos um defeito é encaminhado direto para a lista de descarte.
- **Facilidade para mudar as regras:** Defini os limites de peso e medidas como variáveis no início do script. Se a fábrica mudar o tipo de peça amanhã, basta trocar esses números sem precisar reescrever a lógica interna do programa.

3.2. Dificuldades enfrentadas no desenvolvimento

- **Lógica de abertura e fechamento de caixas:** O ponto que mais me exigiu testes foi o final da produção. Eu precisava garantir que o programa não abrisse uma caixa vazia a mais caso a quantidade total de peças desse exatamente um múltiplo de 10, e também que a última caixa não ficasse perdida no relatório caso terminasse incompleta (por exemplo, com 4 ou 7 peças).
- **Manter a lista de erros limpa:** No início, tive que tomar cuidado para que os motivos de erro de uma peça não fossem acumulados nas peças seguintes, garantindo que cada item tivesse sua avaliação zerada e isolada.

4. Como levar esse projeto para uma indústria de verdade

Embora o código que criei funcione no computador com dados simulados, ele representa o raciocínio lógico que acontece dentro de uma fábrica conectada. Estudando conceitos de Indústria 4.0 e automação inteligente, vejo que para colocar esse sistema em operação real seriam necessários os seguintes passos:

- **Substituir o teclado por sensores físicos (IoT):** Na prática, ninguém digita peso ou tamanho na mão. Instalaríamos células de carga diretamente sob a esteira para pesar as peças em movimento e sensores ópticos a laser (Time-of-Flight) para medir o comprimento com precisão milimétrica. Esses sensores mandariam os dados para o computador via redes industriais como MQTT, Modbus ou OPC-UA.
- **Aplicar visão computacional com IA:** Para a cor e o acabamento da peça, uma câmera industrial com modelos de visão computacional (redes neurais convolucionais) seria a solução ideal. Além de identificar a cor independentemente da iluminação do galpão, a câmera conseguiria achar rachaduras, arranhões ou bolhas na peça, algo que sensores comuns não pegam.

- **Acionamento com CLP e atuadores:** A decisão tomada pelo programa em Python seria enviada a um Controlador Lógico Programável (CLP). O CLP acionaria um pistão pneumático na esteira para empurrar a peça rejeitada para uma caixa de sucata, ou comandaria um braço robótico para organizar as peças boas dentro das caixas.
- **Integração com sistemas de gestão (MES/ERP):** Em vez de apenas exibir no terminal, as informações seriam enviadas diretamente para sistemas corporativos (como SAP ou TOTVS). Isso permite calcular indicadores de eficiência como o OEE em tempo real e agendar manutenção antes mesmo da máquina quebrar.